Technical Document

1.Chunking strategy

a. Why is Chunking Needed?

Chunking divides large documents into smaller meaningful sections, enabling efficient retrieval, reducing token usage, and improving the accuracy of the RAG system.

b. **Chunk Size**

- **Chunk Size:** 500 characters
- **Chunk Overlap:** 50 characters

c. Chunk Size – Reasoning

- **Chunk Size (500 characters):** A size of 500 characters creates smaller, focused chunks that improve retrieval precision and reduce token usage while still preserving enough context for semantic understanding.

- **Chunk Overlap (50 characters):** An overlap of 50 characters maintains continuity between consecutive chunks, ensuring that important information near chunk boundaries is not lost during retrieval.

d. Use of Trade-Off

Smaller chunks improve retrieval accuracy, while larger chunks provide better context but increase token usage.

e. What do the Embedding and Vector DB

Text chunks are converted into embeddings and stored in a vector database for efficient semantic search.

f. Which Vector Database Used

FAISS (Facebook AI Similarity Search) is used as the vector database for storing embeddings and performing efficient similarity searches in the RAG system.

g. How does the retrieval pipeline work in your RAG system?

The retrieval pipeline converts the user query into an embedding, performs similarity search in the vector database, retrieves relevant chunks, and provides them to the LLM.

h. How is similarity search performed in the vector database?

The query embedding is compared with stored embeddings in **FAISS** using cosine similarity to find the most relevant chunks.

i. How are the top-k relevant chunks retrieved?

The system retrieves the top 5 chunks with the highest similarity scores from the vector database.

j. What is Similarity Search?

Similarity search finds the most relevant information by comparing vector embeddings of the query and stored documents.

k. How are similarity scores displayed?

Each retrieved chunk is accompanied by its similarity score, indicating how closely it matches the user query

h. Difference between FIASS , ChromaDB,Pipecone,Weaviate

Search Capability Comparison

FAISS

- Supports high-speed vector similarity search.
- Does not support keyword search.

- Does not provide hybrid search functionality.
- Suitable for local Retrieval-Augmented Generation applications.

ChromaDB

- Supports vector search and metadata filtering.
- Limited support for hybrid search.
- Easy to integrate with LangChain.
- Suitable for prototypes and small-scale applications.

Pinecone

- Supports vector search and hybrid search.
- Managed cloud service with automatic scaling.
- Provides metadata filtering and production-grade performance.
- Suitable for enterprise AI systems.

Weaviate

- Supports vector search, keyword search, and hybrid search.
- Built-in semantic capabilities.
- Supports GraphQL APIs and metadata filtering.
- Suitable for large semantic search applications.

Recommendation for Data Scale (<1 Million Vectors)

FAISS

- Best choice for local deployments.
- Lightweight and highly efficient.
- No external server required.

ChromaDB

- Suitable for rapid prototyping and LangChain-based applications.
- Easy persistence and management.

Pinecone

- Best option for production and cloud-native applications.

- Excellent scalability and managed infrastructure.

Weaviate

- Suitable for applications requiring hybrid search and semantic understanding.